AHMADU BELLO UNIVERSITY, ZARIA

FACULTY OF ENGINEERING

DEPARTMENT OF COMPUTER ENGINEERING

COEN838: ADVANCED SOFTWARE ENGINEERING

Project Examination, Question 4

OFFLINE-FIRST CRYPTOGRAPHIC QR IDENTITY WALLET

FOR RURAL PUBLIC SERVICES

SOFTWARE REQUIREMENTS SPECIFICATION

Document QRW-SRS-01 · Version 1.0

BY

ALIYU SARKI ILIYASU

P25EGCP8040

November 2026

DOCUMENT CONTROL

Document details

Item	Detail
Document	Software Requirements Specification (QRW-SRS-01)
Project	Offline-First Cryptographic QR Identity Wallet for Rural Public Services
Course	COEN838 Advanced Software Engineering, Ahmadu Bello University, Zaria
Author	Aliyu Sarki Iliyasu (P25EGCP8040)
Version / date	1.0 · November 2026
Source code	https://github.com/sarki94/qr-identity-wallet
Related documents	QRW-SDD-02 Design · QRW-SEC-03 Security · QRW-TST-04 Test Plan & Report · QRW-PRM-05 Project, Risk & Maintenance · QRW-USR-06 User Manual · QRW-TR-07 Technical Report

Revision history

Version	Date	Change
0.1	22 Oct 2026	Requirements draft and elicitation
0.2	25 Oct 2026	Localised to Kura LGA; two assurance levels (D2a)
1.0	4 Nov 2026	Complete SRS with measured NFRs and generated RTM

TABLE OF CONTENTS

1. INTRODUCTION....1
1.1 Purpose....1
1.2 Problem Statement and Scope....1
1.3 Definitions, Acronyms and Abbreviations....2
1.4 Stakeholders....3
1.5 Requirements Elicitation....3
1.5.1 Choice of technique....3
1.5.2 Sources and the as-is process....4
1.5.3 Findings and derived requirements....5
1.6 References....5
1.7 Overview of the Document....6
2. GENERAL OVERVIEW....7
2.1 Product Perspective....7
2.2 Product Functions....7
2.3 User Characteristics....8
2.4 Operating Environment....8
2.5 Design and Implementation Constraints....8
2.6 Assumptions and Dependencies....9
3. SPECIFIC REQUIREMENTS....10
3.1 Functional Requirements....10
3.1.1 Enrolment and issuance....10
3.1.2 Trust and revocation....10
3.1.3 Citizen wallet....10
3.1.4 Verifier....11
3.1.5 Audit and synchronisation....11
3.1.6 Staff access....12
3.1.7 Formal specification as state transitions....12
3.2 Non-Functional Requirements....15
3.2.1 Performance....15
3.2.2 Security....16
3.2.3 Usability....16
3.2.4 Capacity and space....17

3.3 Logical Data Requirements....17
4. SYSTEM FEATURES....19
4.1 Enrolment and Issuance....20
4.2 Citizen Wallet....21
4.3 Offline Verification....21
4.4 Revocation and Synchronisation....23
4.5 Audit....23
4.6 Staff Access Management....24
5. EXTERNAL INTERFACE REQUIREMENTS....25
5.1 User Interfaces....25
5.2 Hardware Interfaces....26
5.3 Software Interfaces....26
5.4 Communication Interfaces....27
6. SOFTWARE LIFE CYCLE CONSIDERATIONS....28
6.1 Development Methodology....28
6.2 Version Control....28
6.3 Continuous Integration....28
6.4 Requirements Traceability....28
APPENDIX A: REQUIREMENTS TRACEABILITY MATRIX....30

1. INTRODUCTION

1.1 Purpose

This Software Requirements Specification (SRS) defines the requirements of the Offline-First Cryptographic QR Identity Wallet, the solution developed for Question 4 of the COEN838 Advanced Software Engineering project examination. It states what the system shall do, the qualities it shall have and the constraints under which it operates. It is the baseline against which the design (QRW-SDD-02), the threat model (QRW-SEC-03) and the tests (QRW-TST-04) are traced. The structure follows the six-section SRS template presented in the course: introduction, general overview, specific requirements, system features, external interface requirements and software life cycle considerations.

The intended readers are the course examiners, a programme sponsor such as a State Ministry of Agriculture, and the developer responsible for building and maintaining the system.

1.2 Problem Statement and Scope

The examination question states the problem as follows: rural citizens in Nigeria often cannot access public services, such as local clinics and agricultural subsidies, because online National Identification Number (NIN) and Bank Verification Number (BVN) verification fails due to poor network connectivity.

The system addresses this by moving trust off the network. An enrolment office issues each citizen a credential that is digitally signed by the issuing authority and carried as a QR code, either in a smartphone wallet or on a printed card. At the point of service, a verifier application checks the signature, expiry, revocation status and, for smartphone wallets, proof that the person holds the private key bound to the credential. All of these checks run with no network connection. Records of each check are queued on the verifier's phone and uploaded to a tamper-evident audit log when connectivity returns.

The primary case study is a farmer collecting subsidised fertiliser at a ward distribution point in Kura Local Government Area (LGA), Kano State. Clinic attendance is a secondary use case that uses the same mechanism.

In scope: farmer enrolment against the NIN register; issuance, revocation and offline verification of identity credentials; smartphone (L2) and paper (L1) wallets; staff authentication and role-based access; and an audit trail of verifications.

Out of scope: entitlement management, meaning how many bags a farmer may collect or whether a collection has already taken place. This belongs to subsidy-distribution systems such as the one in examination Question 3. The system answers "is this the real Musa?", not "how much fertiliser does Musa get?". Live integration with the National Identity Management Commission (NIMC) is also out of scope; a mock NIMC service stands in for it (Section 2.6).

1.3 Definitions, Acronyms and Abbreviations

The terms used throughout this document are defined in Table 1.1.

Table 1.1: Definitions and abbreviations

Term	Meaning
Credential	A signed statement by the Issuer of a farmer's identity details (masked NIN, name, ward, LGA, state, validity dates and, for L2, the wallet's public key).
L1 / L2	Assurance levels. L1: printed QR card (paper wallet), checked by signature and the agent's visual comparison. L2: smartphone wallet, additionally proved by challenge–response with a device key.
Holder binding	Proof that the person presenting a credential controls the private key bound into it.
Challenge–response	The verifier shows a fresh random number (nonce); the wallet signs it together with the credential ID; the verifier checks the signature.
Revocation list	A signed, versioned list of credential IDs that are no longer valid.
Offline-first	Designed so that the core task works with no network; the network only refreshes data when available.
PWA	Progressive Web App: a web application installable on a phone and able to run offline.
JWT	JSON Web Token: a signed token carrying a user's identity and role.
Ed25519	The Edwards-curve digital signature algorithm defined in RFC 8032 (Josefsson & Liusvaara, 2017).
Base45	Encoding defined in RFC 9285 that maps binary data onto the QR alphanumeric character set (Fältström et al., 2022).
NIN, BVN, NIMC	National Identification Number; Bank Verification Number; National Identity Management Commission.
LGA, ADP	Local Government Area; Agricultural Development Programme.
RTM	Requirements Traceability Matrix.
FR, NFR	Functional requirement; non-functional requirement.
MoSCoW	Priority scheme: Must have (M), Should have (S), Could have (C), Won't have this time (W).

1.4 Stakeholders

The stakeholders and their goals are listed in Table 1.2. Their identifiers (A1–A6) are used in the use-case model in Section 4.

Table 1.2: Stakeholders and their goals

ID	Stakeholder	Role in the system	Main goal
A1	Farmer (holder)	Holds a wallet or paper wallet and presents it	Collect inputs without repeat trips or being turned away
A2	Enrolment officer	LGA agriculture office staff; enrols, issues, revokes	Register genuine farmers once and correctly
A3	Redemption agent	Extension staff at a ward distribution point; verifies farmers, often offline	Fast, unambiguous verdict for a long queue
A4	Programme administrator	State Ministry of Agriculture; manages staff accounts	Control who can issue and verify
A5	Auditor / M&E officer	Reviews verification records	Detect fraud and prove what happened
A6	NIMC (external system)	Authoritative NIN register (mocked)	Confirm that a NIN exists and matches a person

1.5 Requirements Elicitation

1.5.1 Choice of technique

The course covered contextual inquiry, ethnography, questionnaires and use-case analysis, and the examination brief also names Joint Application Development (JAD) and protocol analysis. Their suitability for this problem is assessed in Table 1.3.

Table 1.3: Suitability of elicitation techniques

Technique	Suitability	Reason
Questionnaires / surveys	Low	Low literacy among rural farmers, dispersed respondents, and answers that reflect what respondents believe rather than what they do.
Interviews / contextual inquiry	Medium	Captures officials' view of the process, but workarounds such as proxy attempts and phone sharing are rarely volunteered.
JAD	Low	Requires officers, agents and administrators to meet together; not feasible for a single-candidate project in the examination period.
Ethnography (direct, non-participant observation)	High (primary)	Exposes tacit knowledge: the undocumented practices and workarounds of a real distribution day, which the lecturer identified as the weakness of interview-only elicitation.
Use-case analysis	High (modelling)	Converts observed practice into actors, goals and flows for the UML models.
Protocol analysis (think-aloud)	Medium (validation)	Better suited to evaluating the prototype with agent-role users.

Ethnography, in the form of direct non-participant observation, was therefore adopted as the primary technique. It was supported by document analysis, and use-case analysis was used to model the findings.

1.5.2 Sources and the as-is process

The author observed manual fertiliser distribution in Kura LGA, Kano State, during the 2024 wet season, attending as an observer and taking no part in the distribution. No personal data about individual farmers was recorded. The observation was triangulated with published evaluations of the Growth Enhancement Support (GES) e-wallet scheme, which list network problems among redemption constraints (Nwaobiala & Ubor, 2019), and with reports of current state schemes that distribute to polling-unit level so that "genuine farmers" can collect directly (African Farming, 2026).

In the observed as-is process the farmer queues on the announced day; the agent finds the farmer's name on a printed ward register; the farmer shows an identity document (a NIN slip or voter's card) for visual comparison; where documents are missing a community leader such as the *Mai Unguwa* or a farmer-group representative vouches for the farmer; and the agent records the collection on the paper register, which is later passed to the LGA office. Identity checking is deliberately manual in order to prevent fraud. This process has three weaknesses: trust depends on a network that is often unavailable, visual document

checks can be defeated by borrowed or altered documents, and paper records cannot be audited for later alteration.

1.5.3 Findings and derived requirements

Table 1.4 links each finding to the requirements it produced. Findings F1–F3 and F6–F9 are tacit: they would not appear in a programme policy document and surfaced only through observation.

Table 1.4: Elicitation findings and derived requirements

ID	Finding	Source	Derived requirements
F1	Proxy collection is attempted but discouraged; identity is checked manually to prevent fraud.	Observation	FR-17, FR-19, FR-23
F2	Households share one phone among several adults.	Observation	FR-10, FR-11, NFR-S2
F3	Many farmers own only feature phones or no phone.	Observation	FR-05, FR-20 (L1 paper wallet)
F4	Network failure is a recurring constraint at redemption points.	Literature, observation	FR-13, FR-15, FR-21, NFR-P3
F5	Schemes aim to reach genuine farmers directly and remove middlemen.	Literature	FR-06, FR-26
F6	Hausa is the working language; many farmers read little English.	Observation	NFR-U1, NFR-U2
F7	Paper registers are the fallback record and cannot be checked for later alteration.	Observation	FR-22, FR-24, FR-25, NFR-S6
F8	Distribution points can be far from the LGA office.	Observation	FR-13 (wallet fully offline)
F9	Community vouching is used when documents fail.	Observation	FR-19 (verifiable alternative)

Limitations: the observation was made by a single observer in one LGA during one season and recorded retrospectively. Requirements with a documentary basis (F4, F5) are supported by independent published sources, and the tacit findings are exercised by scenario-based system tests and a planned think-aloud evaluation with agent-role users.

1.6 References

African Farming. (2026). *Katsina launches fertiliser scheme to strengthen 2026 farming season*. https://africanfarming.net/crops/agriculture/katsina-launches-fertiliser-scheme-to-strengthen-2026-farming-season

Fältström, P., Ljunggren, F., & van Gulik, D.-W. (2022). *The Base45 data encoding* (RFC 9285). Internet Engineering Task Force. https://doi.org/10.17487/RFC9285

Federal Republic of Nigeria. (2023). *Nigeria Data Protection Act, 2023*.

ISO/IEC/IEEE. (2018). *Systems and software engineering — Life cycle processes — Requirements engineering* (ISO/IEC/IEEE 29148:2018).

Jones, M., Bradley, J., & Sakimura, N. (2015). *JSON Web Token (JWT)* (RFC 7519). Internet Engineering Task Force. https://doi.org/10.17487/RFC7519

Josefsson, S., & Liusvaara, I. (2017). *Edwards-curve digital signature algorithm (EdDSA)* (RFC 8032). Internet Engineering Task Force. https://doi.org/10.17487/RFC8032

Nwaobiala, C. U., & Ubor, V. U. (2019). Effectiveness of electronic wallet system of Growth Enhancement Support Scheme distribution among arable crop farmers in Imo State, South-Eastern Nigeria. *Scientific Papers Series Management, Economic Engineering in Agriculture and Rural Development, 16*(1).

Sommerville, I. (2016). *Software engineering* (10th ed.). Pearson.

1.7 Overview of the Document

Section 2 describes the product's context, functions, users, environment, constraints and assumptions. Section 3 states the functional and non-functional requirements, the domain model and the formal state-transition specification. Section 4 describes each system feature with its use cases and sequence model. Section 5 defines the external interfaces, and Section 6 the life-cycle, version-control and continuous-integration approach. Appendix A contains the full Requirements Traceability Matrix.

2. GENERAL OVERVIEW

2.1 Product Perspective

The system is a new, self-contained product built from four small back-end services and one installable web application, as shown in Figure 2.1. The Issuer, Trust & Revocation, Audit & Sync and Auth services run on a server; the Issuer Portal, Citizen Wallet and Verifier are screens of a single Progressive Web App that runs on laptops and Android phones. The wallet and the verifier communicate with each other only through QR codes, which is what allows verification without a network.

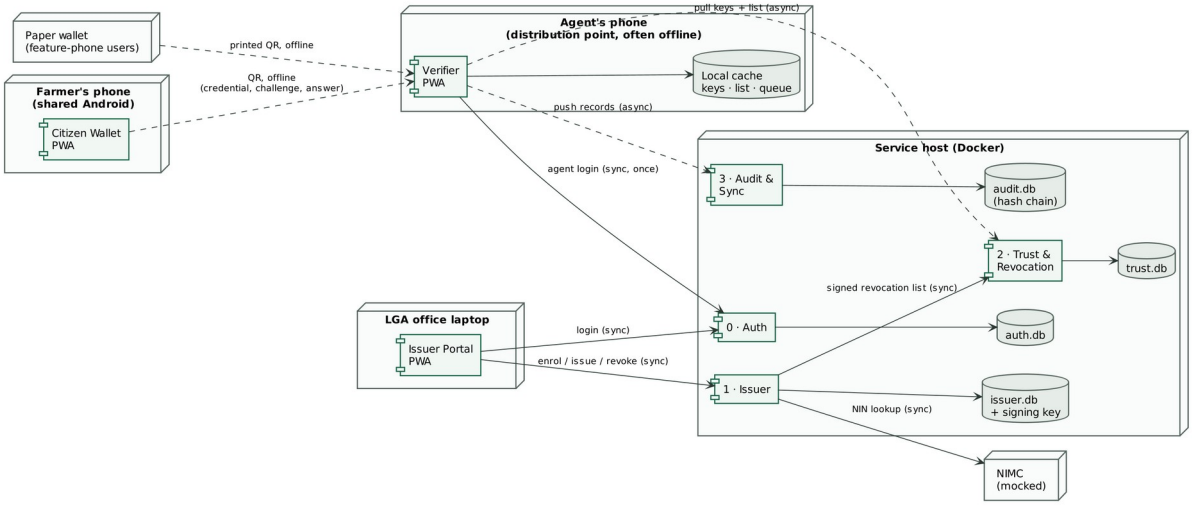


Figure 2.1: System context and service architecture (solid arrows: synchronous calls; dashed arrows: asynchronous or offline exchanges)

2.2 Product Functions

At a high level, the system shall:

- enrol farmers by confirming their NIN with NIMC, storing only a masked NIN and a keyed hash;
- issue Ed25519-signed identity credentials to a smartphone wallet (L2) or as a printed card (L1), and revoke them;
- publish the issuer's public keys and a signed revocation list for verifiers to download;
- let a redemption agent verify a credential, the holder's possession of the device key and the holder's face, entirely offline;
- queue every verification on the verifier and upload it to a hash-chained audit log when connectivity returns;

- authenticate staff and restrict every operation to the appropriate role.

2.3 User Characteristics

The personas in Table 2.1 are illustrative composites built from the elicitation findings; they do not describe real people.

Table 2.1: User personas

Persona	Characteristics	Implications
P1 Malam Musa Ibrahim, farmer (L2)	Aged 48; rice and maize near Kura, Kano. Shares an Android phone with his adult son. Reads Hausa; limited English.	Multiple PIN-protected profiles per phone; icon and colour verdicts.
P2 Hajiya Hauwa Garba, farmer (L1)	Aged 55; groundnut farmer; feature phone only.	Printed paper wallet; agent's visual check.
P3 Malam Bello Sani, redemption agent	Extension staff at a ward distribution point; Android phone; signal intermittent or absent for days.	Offline verification; at most four taps; list-age warnings.
P4 Enrolment officer	Office-based with reliable connectivity.	Online issuance, wallet binding, card printing, revocation.
P5 State M&E officer (auditor)	Reviews records across wards.	Chain check, filters, cloning flags.

2.4 Operating Environment

- **Services:** Python 3.11 or later with FastAPI; each service owns its own database (SQLite in the prototype, PostgreSQL in production); deployable with Docker Compose or the included launcher on Windows, macOS or Linux.
- **Web application:** React 18 and TypeScript, built with Vite as a Progressive Web App; runs in current Chrome or Edge on Android 8 or later and on desktop computers.
- **Phones:** a camera is needed to scan QR codes; HTTPS is required by the browser for the camera and for Web Crypto.
- **Network:** enrolment offices are online; distribution points may be offline for days.

2.5 Design and Implementation Constraints

- Single developer; fixed examination deadline of 8 November 2026.
- Architecture must be distributed or service-oriented (examination brief).

- Cryptography limited to well-established primitives: Ed25519 signatures, SHA-256 hashing, PBKDF2 and AES-GCM in the browser; no custom cryptography.
- Personal data handled according to the principles of the Nigeria Data Protection Act 2023: purpose limitation and data minimisation.
- Low-end Android phones and low literacy among users (Section 2.3).

2.6 Assumptions and Dependencies

As required by Instruction D of the examination, every assumption made in interpreting the problem is stated in Table 2.2. Domain invariants that practitioners treat as obvious, such as the phone clock being right (AS9), are stated explicitly because leaving such tacit knowledge unstated is a common cause of requirements failure.

Table 2.2: Assumptions and dependencies

ID	Assumption	Consequence if false
AS1	NIMC is represented by a mock service with fictional records; production integration requires NIMC licensing.	Issuer must call the real NIMC interface; no change to credentials or verifiers.
AS2	The issuer signing key is held in an encrypted file in the prototype; production would use a hardware security module.	Key compromise risk rises; see risk register.
AS3	L2 holders have an Android phone (own or shared) with a camera; all others use the L1 paper wallet.	More L1 use, lower assurance.
AS4	Agents are given an Android smartphone by the programme.	Verification cannot run at that point.
AS5	Distribution points obtain at least intermittent connectivity within seven days.	Revocations reach them later; the 30-day refusal rule applies.
AS6	Entitlement (quantities and collection status) is managed by a separate programme system.	Scope would extend into Question 3.
AS7	Proxy collection is prohibited by programme policy (observed in Kura).	A delegation feature would be needed.
AS8	Enrolment takes place at an office with connectivity.	Offline issuance would need a different key-protection design.
AS9	The agent's phone shows the correct date (to within a day). Expiry checks and the 7- and 30-day list-age rules use the phone's clock; the demo clock can only move time forward, which makes the rules stricter.	An expired credential or a stale list could be accepted if the clock is set back; planned fix: compare with server time at each sync.

3. SPECIFIC REQUIREMENTS

Each requirement has a unique identifier used in the RTM, a MoSCoW priority and an implementation status. Status is reported honestly: **Implemented**, or **Partial** with the missing part stated.

3.1 Functional Requirements

3.1.1 Enrolment and issuance

The enrolment and issuance requirements are listed in Table 3.1.

Table 3.1: Functional requirements: enrolment and issuance

ID	The system shall…	Priority	Status
FR-01	Officer looks up a farmer by NIN through NIMC before enrolment.	M	Implemented
FR-02	Farmer photo captured and embedded as a small signed thumbnail.	S	Implemented
FR-03	Issuer signs a credential with ID, masked NIN, name, ward, LGA, state, dates, key ID, level and (L2) holder key.	M	Implemented
FR-04	L2: wallet generates a device key; officer binds its public key into the credential.	M	Implemented
FR-05	L1: printable paper-wallet card with signed QR.	S	Implemented
FR-06	At most one active credential per NIN; re-issue revokes the old one atomically (D8).	M	Implemented
FR-07	Officer revokes a credential with a recorded reason.	M	Implemented

3.1.2 Trust and revocation

The trust and revocation requirements are listed in Table 3.2.

Table 3.2: Functional requirements: trust and revocation

ID	The system shall…	Priority	Status
FR-08	Trust service publishes issuer public keys with key IDs.	M	Implemented
FR-09	Signed, versioned revocation list; Trust verifies it and refuses rollback (D11).	M	Implemented

3.1.3 Citizen wallet

The citizen wallet requirements are listed in Table 3.3.

Table 3.3: Functional requirements: citizen wallet

ID	The system shall…	Priority	Status
FR-10	Several farmer profiles on one shared phone, each with its own key and PIN.	S	Implemented
FR-11	Credential shown only after the profile PIN is entered.	M	Implemented
FR-12	Wallet signs (challenge + credential ID) with the device key and shows the answer QR.	M	Implemented
FR-13	All wallet functions work with no network after enrolment.	M	Implemented

3.1.4 Verifier

The verifier requirements are listed in Table 3.4.

Table 3.4: Functional requirements: verifier

ID	The system shall…	Priority	Status
FR-14	Agent logs in online once, then works offline for 7 days. (Agent PIN unlock not built: the 7-day token is kept on the phone.)	M	Partial
FR-15	Verifier downloads keys and revocation list when online and on demand, checking the list signature itself.	M	Implemented
FR-16	Verifier checks the issuer signature offline.	M	Implemented
FR-17	L2: fresh random challenge; wallet answer verified against the holder key.	M	Implemented
FR-18	Verifier checks expiry and revocation against its cached list.	M	Implemented
FR-19	Signed photo or details shown; agent must confirm the face before acceptance.	M	Implemented
FR-20	Verdict shown as Accepted (L2), Accepted (L1) or Rejected with reason.	M	Implemented
FR-21	Warn when the list is older than 7 days; refuse acceptances after 30 days.	M	Implemented
FR-22	Every attempt (accepted or rejected) recorded in a local queue.	M	Implemented
FR-23	No proxy or delegated presentation; the holder must be present (D6).	M	Implemented

3.1.5 Audit and synchronisation

The audit and synchronisation requirements are listed in Table 3.5.

Table 3.5: Functional requirements: audit and synchronisation

ID	The system shall…	Priority	Status
FR-24	Queued records uploaded in batches and appended to a hash chain; agent identity from the token.	M	Implemented
FR-25	Auditor filters records and runs a chain-integrity check.	M	Implemented
FR-26	Flag a credential accepted at two points within 24 hours (possible cloning).	S	Implemented

3.1.6 Staff access

The staff access requirements are listed in Table 3.6.

Table 3.6: Functional requirements: staff access

ID	The system shall…	Priority	Status
FR-27	Staff sign in with username and password. (TOTP MFA for Admin/Auditor deferred: cut list item 2.)	M	Partial
FR-28	Every endpoint enforces role-based access (Officer, Agent, Admin, Auditor).	M	Implemented
FR-29	Admin creates, disables, unlocks and assigns staff to distribution points.	S	Implemented

3.1.7 Formal specification as state transitions

Following the course's formal notation, a functional requirement can be expressed as a state-transition function f: $S \times I \rightarrow S \times O$, where S is the set of system states, I the inputs and O the outputs. The verification behaviour (FR-16 to FR-22) is specified this way in Table 3.7, with S = {Idle, CredentialRead, ChallengeShown, AwaitingFaceCheck, Accepted, Rejected, RecordQueued}.

Table 3.7: Verification behaviour as f: S × I → S × O

Current state (S)	Input (I)	Next state (S)	Output (O)
Idle	Scan credential QR	CredentialRead	Parsed credential
CredentialRead	Signature invalid, expired, revoked, unreadable, or list older than 30 days	Rejected	Reason shown
CredentialRead	Valid and level = L2	ChallengeShown	Challenge QR (fresh nonce)
CredentialRead	Valid and level = L1	AwaitingFaceCheck	Signed details and photo
ChallengeShown	Answer signed by the holder key	AwaitingFaceCheck	Signed details and photo
ChallengeShown	Invalid answer or 60 s timeout	Rejected	"Copy or replay" reason
AwaitingFaceCheck	Agent confirms the face	Accepted	Accepted (L1 or L2)
AwaitingFaceCheck	Agent rejects the face	Rejected	"Face mismatch"
Accepted or Rejected	(automatic)	RecordQueued	Record added to local queue

The same behaviour is drawn as a state machine in Figure 3.1, in the guarded style the lecturer used for the HeartRateMonitor example, and the credential lifecycle (FR-03 to FR-07) in Figure 3.2.

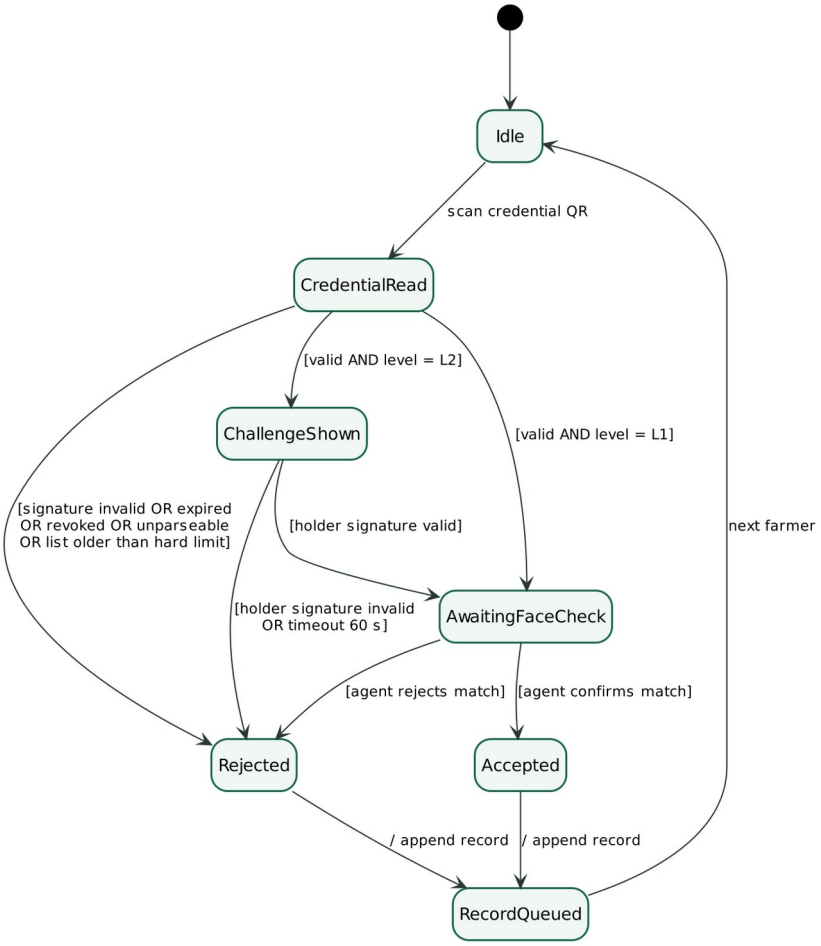


Figure 3.1: State machine of a verification session (guards in square brackets)

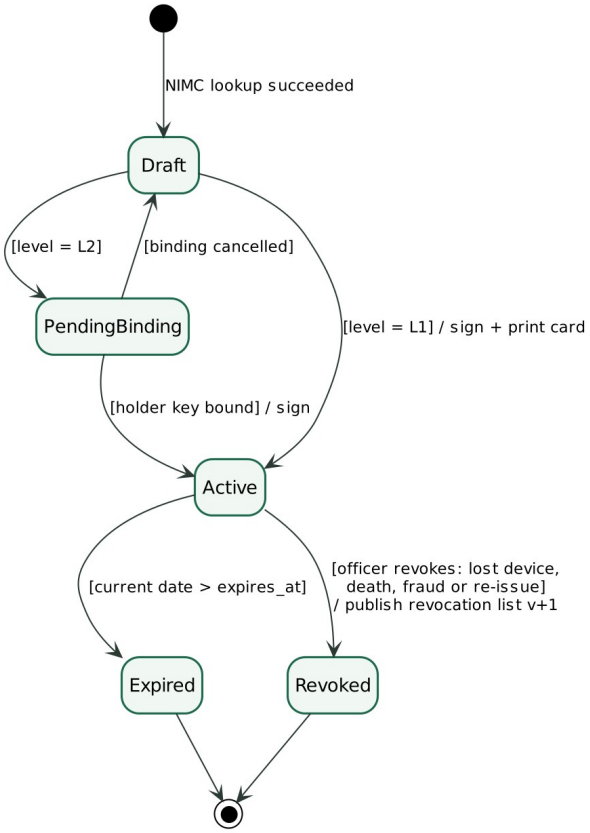


Figure 3.2: State machine of the credential lifecycle

3.2 Non-Functional Requirements

The non-functional requirements are grouped under the four categories used in the course template. Where a requirement is measurable, the value measured by the automated tests on 3 November 2026 is given.

3.2.1 Performance

Table 3.8 lists the performance requirements.

Table 3.8: Performance requirements

ID	Requirement	Priority	Status	Measured
NFR-P1	Offline verification (scan to verdict, excluding people) within 3 s on a low-end phone.	M	Implemented	0.4 ms (Python), 5.2 ms (browser library) per full check
NFR-P2	Credential issuance completes within 5 s.	M	Implemented	≈ 10 ms per issuance
NFR-P3	Core verification available with zero connectivity; services degrade gracefully when one is down.	M	Implemented	—

3.2.2 Security

Table 3.9 lists the security requirements.

Table 3.9: Security requirements

ID	Requirement	Priority	Status	Measured
NFR-S1	Ed25519 signing; issuer private key only in the Issuer, encrypted at rest (HSM in production).	M	Implemented	—
NFR-S2	Holder private keys sealed with a PIN-derived key (PBKDF2 + AES-GCM).	M	Implemented	—
NFR-S3	TLS on all traffic. (Prototype: HTTPS between phones and the app; services on localhost behind the proxy. Production: TLS reverse proxy.)	M	Partial	—
NFR-S4	Data minimisation: masked NIN only; full NIN never stored (keyed hash, D13).	M	Implemented	—
NFR-S5	Fail securely: any error, unknown key or bad input is a rejection, never a crash or leak.	M	Implemented	—
NFR-S6	Audit records tamper-evident (hash chain).	M	Implemented	—
NFR-S7	Login rate-limited; account locks after 5 failed logins.	M	Implemented	—
NFR-S8	Nigeria Data Protection Act 2023 principles. (Minimisation and purpose limitation built in; consent capture is a procedural step at enrolment.)	M	Partial	—

3.2.3 Usability

Table 3.10 lists the usability requirements.

Table 3.10: Usability requirements

ID	Requirement	Priority	Status	Measured
NFR-U1	Hausa and English. (Verdict strings in Hausa; full translation deferred.)	S	Partial	—
NFR-U2	Verdicts readable without reading English: colour, icon, Hausa word.	M	Implemented	—
NFR-U3	An L2 verification takes at most 4 agent taps.	M	Implemented	—
NFR-U4	Usable on 5-inch screens; touch targets at least 48 px.	M	Implemented	—

3.2.4 Capacity and space

Table 3.11 lists the capacity and space requirements.

Table 3.11: Capacity and Space requirements

ID	Requirement	Priority	Status	Measured
NFR-C1	Credential QR at most 1,800 bytes, including a photo thumbnail.	M	Implemented	Within 1,800 bytes with a 1 KB photo
NFR-C2	Wallet stores up to 5 profiles in at most 5 MB.	S	Implemented	3.4 KB for five profiles
NFR-C3	Revocation list of 100,000 entries transferred and cached in at most 2 MB (compressed). First test failed at 2.40 MB uncompressed; fixed with gzip in transit and deflate in the phone cache.	M	Implemented	0.94 MB in transit, 1.41 MB cached
NFR-C4	At least 2,000 queued records per upload.	M	Implemented	2,000 records accepted in one batch

3.3 Logical Data Requirements

The domain model in Figure 3.3 shows the information the system manages, grouped by the service that owns it. Following the course's emphasis on domain integrity, composition expresses that a Credential cannot exist without its Farmer, who must first have been confirmed by NIMC, in the same way that the lecturer's example required a Customer to exist before an Order. Three integrity rules apply: a credential can only be created for an enrolled farmer; at most one credential per farmer is active; and an L2 credential carries exactly one holder key.

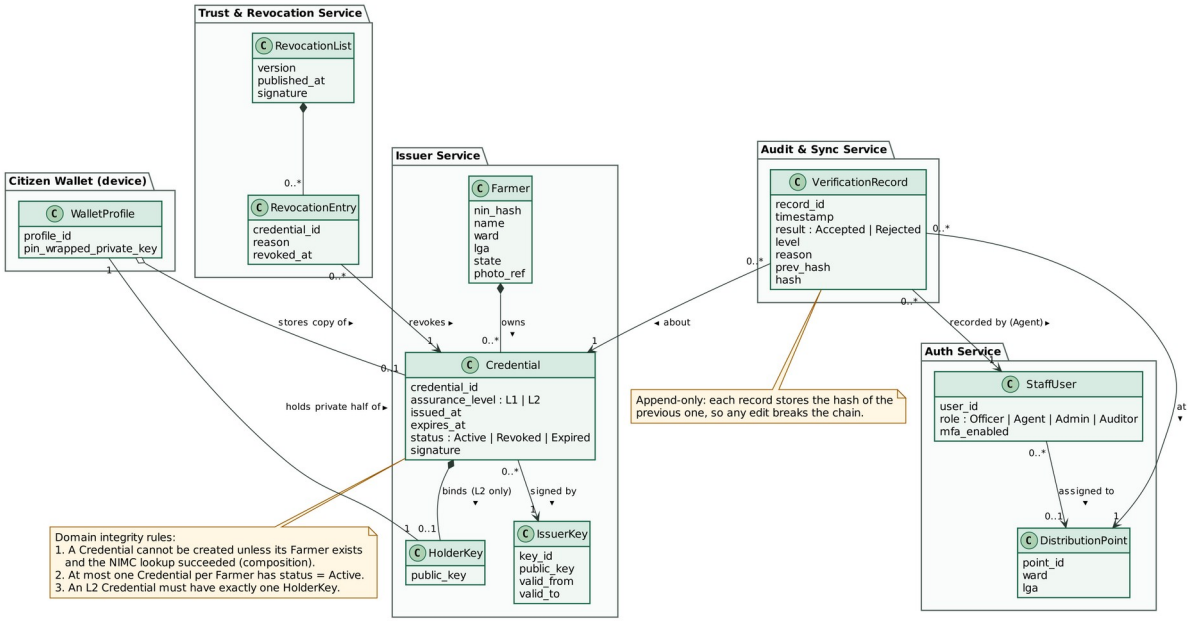


Figure 3.3: Domain model (class diagram) by owning service

4. SYSTEM FEATURES

The system's features are organised around the use cases in Figure 4.1. Each use case carries an identifier that appears in the RTM.

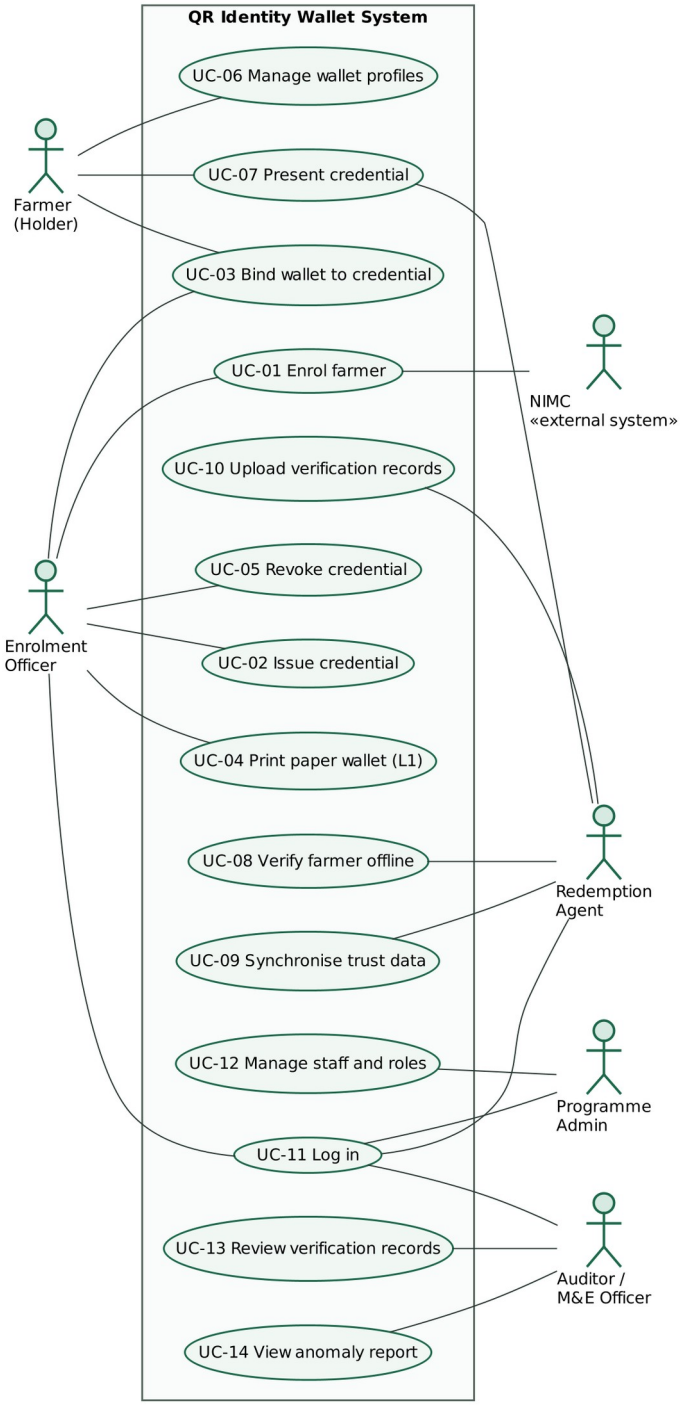


Figure 4.1: Use-case diagram

4.1 Enrolment and Issuance

Description and priority. Must have. An enrolment officer confirms a farmer's identity with NIMC and issues a signed credential to the farmer's wallet or as a paper wallet. Issuing again for the same farmer revokes the earlier credential in the same database transaction.

Stimulus and response. The officer enters a NIN; the system returns the NIMC record. The officer scans the wallet's binding code (L2) or chooses a paper wallet (L1); the system returns a signed credential QR. The interaction is shown in Figure 4.2.

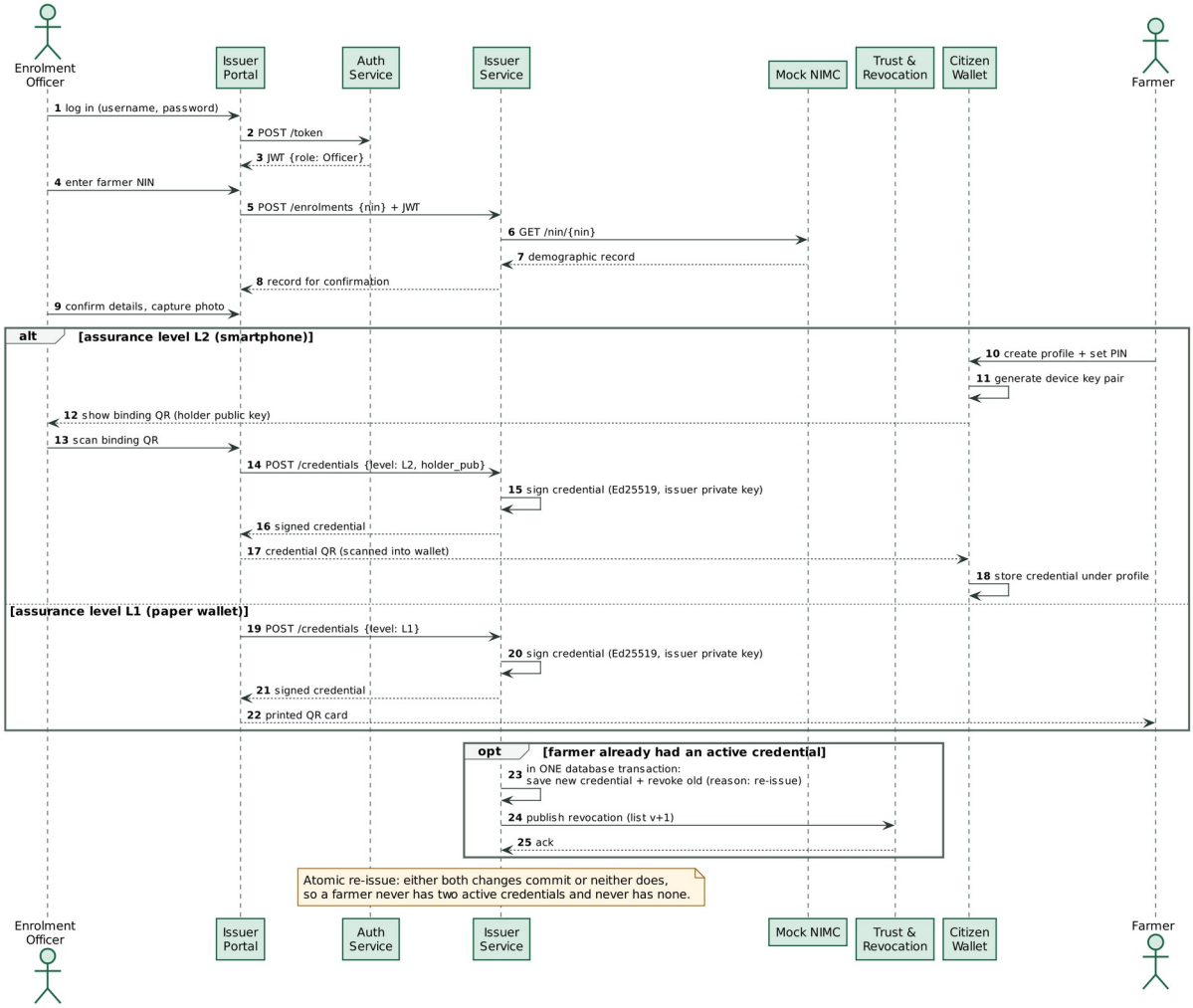


Figure 4.2: Sequence diagram: enrolment and credential issuance (synchronous)

Table 4.1: Use case UC-01/02/03: Enrol farmer, issue credential, bind wallet

Element	Description
Actors	Enrolment officer (primary); farmer; NIMC (secondary)
Preconditions	Officer signed in; NIMC reachable.
Main flow	1. Officer enters NIN. 2. System looks up NIMC and shows the record. 3. Farmer creates a wallet profile with a PIN; wallet shows binding code. 4. Officer scans it. 5. System signs and returns the credential. 6. Farmer scans it into the wallet.
Alternative flows	2a. NIN not found: system reports it and stops. 3a. Farmer has no smartphone: officer selects L1 and prints the paper wallet (UC-04). 5a. Farmer already has an active credential: it is revoked in the same transaction and a new revocation list is published.
Postconditions	One active credential for the farmer; revocation list updated if a previous one existed.
Requirements	FR-01 to FR-06

4.2 Citizen Wallet

Description and priority. Must have. The wallet holds one or more farmer profiles on a shared phone. Each profile has its own device key pair, sealed with its own six-digit PIN, and its own credential. All wallet functions work without a network.

Table 4.2: Use case UC-06/07: Manage profiles and present credential

Element	Description
Actors	Farmer
Preconditions	Credential saved in the profile.
Main flow	1. Farmer selects the profile and enters the PIN. 2. Wallet shows the credential QR. 3. Agent shows a challenge; farmer scans it. 4. Wallet signs the challenge with the device key and shows the answer QR.
Alternative flows	1a. Wrong PIN: remaining attempts shown; after five failures the profile is locked for five minutes. 3a. Challenge is for a different credential: wallet refuses.
Postconditions	Agent holds the signed answer; the private key never leaves the phone.
Requirements	FR-10 to FR-13

4.3 Offline Verification

Description and priority. Must have. This is the core feature required by the problem statement. The agent's phone verifies the credential, the holder and the face using only data downloaded at its last synchronisation. The sequence is shown in Figure 4.3.

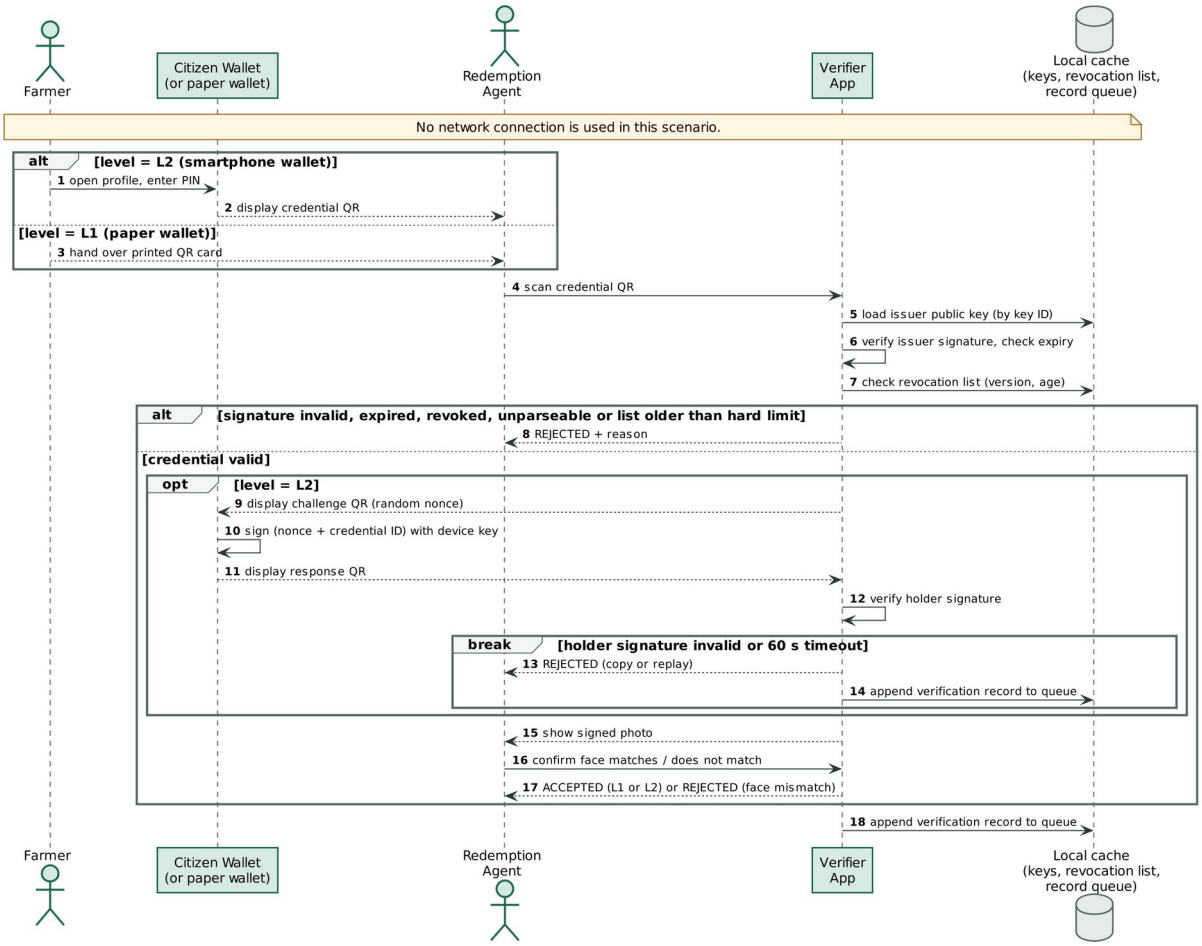


Figure 4.3: Sequence diagram: offline verification at a distribution point

Table 4.3: Use case UC-08: Verify farmer offline

Element	Description
Actors	Redemption agent (primary); farmer
Preconditions	Verifier has synchronised at least once and its revocation list is less than 30 days old.
Main flow	1. Agent scans the credential QR. 2. Verifier checks signature, expiry and revocation. 3. For L2, verifier shows a challenge and scans the wallet's answer. 4. Verifier shows the signed details and photo. 5. Agent confirms the face. 6. Verifier shows ACCEPTED and queues a record.
Alternative flows	2a. Any check fails: REJECTED with the reason. 3a. Answer invalid or late: REJECTED (copy or replay). 5a. Face does not match: REJECTED. 0a. List older than seven days: warning; older than 30 days: no acceptance until synchronised.
Postconditions	A verification record is queued.
Requirements	FR-16 to FR-23

4.4 Revocation and Synchronisation

Description and priority. Must have. The officer revokes a credential; the Issuer signs a new revocation list and pushes it to the Trust service. Verifiers download keys and the list, and upload their queued records, whenever they have a signal (Figure 4.4).

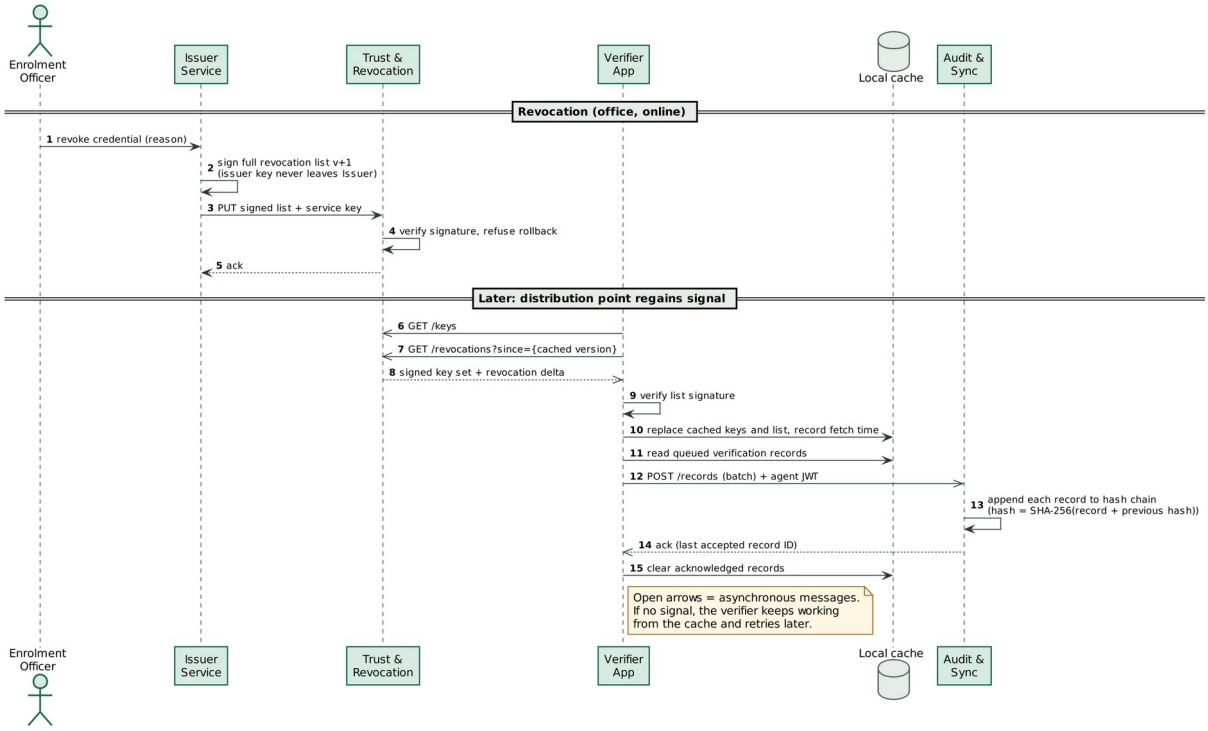


Figure 4.4: Sequence diagram: revocation and asynchronous synchronisation

Table 4.4: Use case UC-05/09/10: Revoke, synchronise, upload records

Element	Description
Actors	Enrolment officer; redemption agent
Preconditions	Officer online; agent's phone regains signal.
Main flow	1. Officer revokes with a reason. 2. Issuer signs list version n+1 and pushes it to Trust, which verifies it. 3. Verifier downloads keys and the list and checks the signature. 4. Verifier uploads queued records; Audit chains them.
Alternative flows	2a. Trust unreachable: revocation is stored and republished later. 3a. Older list offered: refused. 4a. Upload interrupted: retried; duplicates are ignored.
Postconditions	Verifier refuses the revoked credential from its next check.
Requirements	FR-07 to FR-09, FR-15, FR-24

4.5 Audit

Description and priority. Must have (FR-26 Should have). The auditor reviews all verification records, filters them, checks the integrity of the hash chain and sees credentials accepted at two distribution points within 24 hours.

4.6 Staff Access Management

Description and priority. Must have. Staff sign in with a username and password and receive a signed token carrying their role and, for agents, their distribution point. Every service enforces role-based access. Administrators create, disable and unlock accounts. Agent tokens last seven days so that verifiers can work offline.

5. EXTERNAL INTERFACE REQUIREMENTS

5.1 User Interfaces

The four screens of the web application and their users are listed in Table 5.1. Verdicts use colour, an icon and a Hausa word ("An karɓa" for accepted, "An ƙi" for rejected) so that they can be read without English (NFR-U2), and touch targets are at least 48 pixels high (NFR-U4). Figure 5.1 shows the verifier's accepted verdict.

Table 5.1: User interface screens

Screen	Device	User	Network
Issuer Portal	Laptop at LGA office	Enrolment officer	Required
Citizen Wallet	Farmer's (shared) Android phone	Farmer	Not required after enrolment
Verifier	Agent's Android phone	Redemption agent	Only to synchronise
Audit	Laptop	Auditor	Required
Guided Demo	Laptop or projector	Presenter	Required (calls live services)

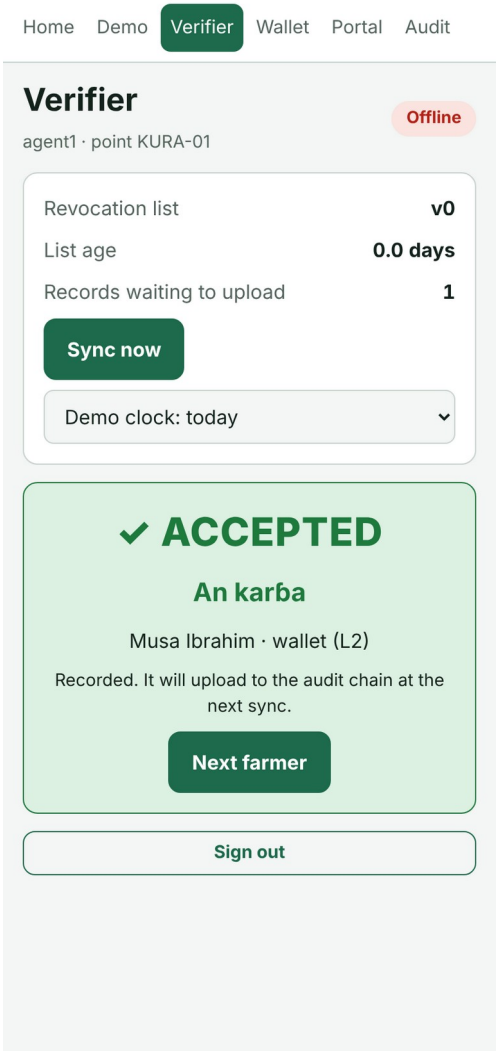


Figure 5.1: Verifier screen showing an offline acceptance

5.2 Hardware Interfaces

- Phone or laptop camera for QR scanning, accessed through the browser; a paste fallback exists for devices without a camera.
- Any printer for L1 paper wallets, through the browser's print function.
- Optional camera at the enrolment office for the farmer's photo thumbnail.

5.3 Software Interfaces

The services expose REST interfaces with JSON bodies. The main operations are summarised in Table 5.2; each service also publishes interactive OpenAPI documentation at /docs.

Table 5.2: Service interfaces

Service (port)	Operation	Caller / role
Auth (8001)	POST /token; GET /me; GET /public-key; POST, GET, PATCH /users	All staff; Admin for /users
Issuer (8002)	POST /enrolments; POST /credentials; POST /credentials/{id}/revoke; POST /revocations/publish; GET /farmers/{id}/credentials	Officer (Admin for republish and listing)
Trust & Revocation (8003)	GET /keys; GET /revocations; PUT /internal/revocations	Anyone (public data); Issuer with service key
Audit & Sync (8004)	POST /records; GET /records; GET /records/verify-chain; GET /anomalies	Agent (upload); Auditor and Admin
Mock NIMC (8005)	GET /nin/{nin}	Issuer only

5.4 Communication Interfaces

Between phones and the server, all traffic uses HTTPS through a single origin, with requests proxied to the services. Between the wallet and the verifier there is no network: the devices exchange the four QR formats in Table 5.3.

Table 5.3: Device-to-device QR formats

Prefix	Carries	From → to
QRW1:	Base45 of zlib(signature ‖ credential JSON)	Wallet or paper wallet → verifier; portal → wallet
QRW1B:	Wallet's public key (binding code)	Wallet → issuer portal
QRW1C:	Credential ID and fresh nonce (challenge)	Verifier → wallet
QRW1R:	Credential ID and signature over the challenge (answer)	Wallet → verifier

6. SOFTWARE LIFE CYCLE CONSIDERATIONS

6.1 Development Methodology

An iterative, Agile-style approach was chosen over Waterfall. The requirements were stable in their core (offline verification) but uncertain in detail, as the change from a smartphone-only wallet to two assurance levels after elicitation showed. A solo developer working to a fixed deadline also needed working software early, because 45 of the 100 marks depend on a running system. Work was planned as daily increments with story points, which provide the earned-value figures reported in QRW-PRM-05. The cryptographic core was built test-first (TDD), and its Git history shows the failing tests committed before the implementation.

6.2 Version Control

All source code, UML models (as PlantUML text) and documentation are kept in a single Git repository hosted on GitHub. Commits follow a conventional-commit style (feat, fix, test, docs, security). Commits made with AI assistance carry a Co-Authored-By trailer, in line with Instruction F on the responsible use of AI.

6.3 Continuous Integration

Every push triggers the GitHub Actions jobs listed in Table 6.1.

Table 6.1: Continuous-integration jobs

Job	Steps
Core	Flake8 lint; Bandit static analysis; pytest with Hypothesis property tests; 90 % coverage gate
Services	Flake8; Bandit; pip-audit dependency scan; API, security, integration and system tests; 90 % coverage gate
Web	npm audit of shipped packages; TypeScript strict type-check; Vitest cross-language tests with coverage; production build
Docker	Builds the service image

6.4 Requirements Traceability

The RTM is generated from the code by tools/rtm.py: every test is tagged with the requirement identifiers it verifies, and the tool combines those tags with the results of an actual test run. On 3 November 2026 the matrix covered all 48 requirements, with no

failing tests and no unverified requirements. The full matrix is given in Appendix A in the lecturer's four-column format.

APPENDIX A: REQUIREMENTS TRACEABILITY MATRIX

Format: the lecturer's Traceability Verification Matrix. Generated by tools/rtm.py on 3 November 2026; regenerate after any change.

Table A.1: Requirements Traceability Matrix

Requirement ID	UML / SysML artefact	Code module	Test case ID (result)
FR-01 (M, Implemented)	UC diagram (Fig. 4.1) UC-01; SD-1 Issuance sequence (Fig. 4.2)	issuer/app.py enrol()	TC-SEC-002 (Pass); TC-SVC-020 (Pass); TC-SYS-001 (Pass); TC-SYS-003 (Pass)
FR-02 (S, Implemented)	Class diagram (Fig. 3.3) Credential.photo	web Portal.tsx photoToB64(); qrw_core.credential	TC-CORE-022 (Pass)
FR-03 (M, Implemented)	SD-1 Issuance sequence (Fig. 4.2); SM-2 Credential lifecycle (Fig. 3.2)	qrw_core.credential.issue_credential(); issuer/app.py issue()	TC-CORE-010 (Pass); TC-CORE-020 (Pass); TC-CORE-061 (Pass); TC-SVC-021 (Pass); TC-SVC-040 (Pass); TC-SYS-001 (Pass); TC-SYS-003 (Pass)
FR-04 (M, Implemented)	SD-1 Issuance sequence (Fig. 4.2); SM-2 Credential lifecycle (Fig. 3.2) PendingBinding	web qrw.ts newHolderKey(); issuer/app.py issue()	TC-CORE-012 (Pass); TC-CORE-061 (Pass); TC-SVC-021 (Pass); TC-SVC-022 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass)
FR-05 (S, Implemented)	UC diagram (Fig. 4.1) UC-04; SM-2 Credential lifecycle (Fig. 3.2)	web Portal.tsx (print); qrw_core.credential	TC-CORE-011 (Pass); TC-SVC-022 (Pass)
FR-06 (M, Implemented)	SD-1 Issuance sequence (Fig. 4.2) opt re-issue; Class diagram (Fig. 3.3) rule 2	issuer/app.py issue() single transaction	TC-SVC-023 (Pass); TC-SYS-003 (Pass)
FR-07 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4); SM-2 Credential lifecycle (Fig. 3.2) Revoked	issuer/app.py revoke()	TC-SVC-024 (Pass); TC-SVC-026 (Pass); TC-SYS-001 (Pass); TC-SYS-003 (Pass)
FR-08 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4)	trust/app.py keys()	TC-SVC-021 (Pass)
FR-09 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4); Class diagram (Fig. 3.3) RevocationList	qrw_core.revocation; trust/app.py receive()	TC-CORE-040 (Pass); TC-CORE-041 (Pass); TC-CORE-042 (Pass); TC-CORE-071 (Pass); TC-SVC-023 (Pass); TC-SVC-024 (Pass); TC-SVC-025 (Pass); TC-SYS-003 (Pass); TC-WEB-008 (Pass)
FR-10 (S, Implemented)	UC diagram (Fig. 4.1) UC-06	web Wallet.tsx	TC-SYS-002 (Pass); TC-WEB-015 (Pass)
FR-11 (M, Implemented)	UC diagram (Fig. 4.1) UC-07	web Wallet.tsx unlock(); vault.ts	TC-SYS-002 (Pass); TC-WEB-014 (Pass)
FR-12 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3)	qrw_core.challenge.sign_challenge(); web qrw.ts signChallenge()	TC-CORE-030 (Pass); TC-SYS-002 (Pass); TC-WEB-006 (Pass)
FR-13 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3)	web Wallet.tsx (no API calls); PWA service worker	TC-SYS-002 (Pass)
FR-14 (M, Partial)	UC diagram (Fig. 4.1) UC-11	auth/app.py token(); web Verifier.tsx	TC-SEC-005 (Pass); TC-SVC-011 (Pass); TC-SYS-003 (Pass)
FR-15 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4)	web Verifier.tsx sync(); qrw.ts verifyRevocationList()	TC-SVC-042 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass); TC-SYS-003 (Pass); TC-WEB-008 (Pass)
FR-16 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3); SM-1 Verification state machine (Fig. 3.1) CredentialRead	qrw_core.credential.verify_credential(); web qrw.ts verifyCredential()	TC-CORE-010 (Pass); TC-CORE-013 (Pass); TC-CORE-014 (Pass); TC-CORE-020 (Pass); TC-CORE-021 (Pass); TC-SVC-021 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass); TC-SYS-003 (Pass); TC-WEB-002 (Pass); TC-WEB-003 (Pass); TC-WEB-012 (Pass)

Requirement ID	UML / SysML artefact	Code module	Test case ID (result)
FR-17 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3); SM-1 Verification state machine (Fig. 3.1) ChallengeShown	qrw_core.challenge; web qrw.ts verifyChallengeResponse()	TC-CORE-030 (Pass); TC-CORE-031 (Pass); TC-CORE-032 (Pass); TC-CORE-033 (Pass); TC-CORE-034 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass); TC-SYS-003 (Pass); TC-WEB-006 (Pass); TC-WEB-009 (Pass)
FR-18 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1)	qrw_core.credential.verify_credential()	TC-CORE-016 (Pass); TC-CORE-017 (Pass); TC-CORE-062 (Pass); TC-SVC-024 (Pass); TC-SYS-001 (Pass); TC-SYS-003 (Pass); TC-WEB-004 (Pass)
FR-19 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1) AwaitingFaceCheck	web Verifier.tsx face step	TC-SYS-002 (Pass); Demo: Verifier face-check step (TC-SYS-002 screenshot 08) (Pass)
FR-20 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1)	web Verifier.tsx	TC-SYS-002 (Pass); Demo: TC-SYS-002 screenshots 09-13 (Pass)
FR-21 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1) guard	web Verifier.tsx SOFT_DAYS/HARD_DAYS	Demo: Verifier demo clock +8 / +31 days (Pass)
FR-22 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1) RecordQueued	web Verifier.tsx finish()	TC-SVC-030 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass)
FR-23 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3)	qrw_core.challenge; Verifier face check	TC-CORE-031 (Pass); TC-CORE-032 (Pass); TC-SYS-002 (Pass); TC-WEB-007 (Pass)
FR-24 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4)	audit/app.py upload(); qrw_core.audit	TC-CORE-050 (Pass); TC-SEC-006 (Pass); TC-SVC-030 (Pass); TC-SVC-041 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass); TC-SYS-003 (Pass)
FR-25 (M, Implemented)	UC diagram (Fig. 4.1) UC-13	audit/app.py check_chain(); qrw_core.audit.verify_chain()	TC-CORE-050 (Pass); TC-CORE-051 (Pass); TC-CORE-052 (Pass); TC-CORE-053 (Pass); TC-SVC-031 (Pass); TC-SVC-032 (Pass); TC-SYS-001 (Pass); TC-SYS-002 (Pass); TC-SYS-003 (Pass)
FR-26 (S, Implemented)	UC diagram (Fig. 4.1) UC-14	audit/app.py anomalies()	TC-SVC-033 (Pass)
FR-27 (M, Partial)	UC diagram (Fig. 4.1) UC-11; SD-1 Issuance sequence (Fig. 4.2)	auth/app.py token()	TC-SEC-001 (Pass); TC-SVC-010 (Pass)
FR-28 (M, Implemented)	Class diagram (Fig. 3.3)	shared/security.py require_roles()	TC-SEC-004 (Pass); TC-SEC-005 (Pass); TC-SVC-013 (Pass); TC-SVC-014 (Pass)
FR-29 (S, Implemented)	UC diagram (Fig. 4.1) UC-12	auth/app.py users endpoints	TC-SVC-012 (Pass); TC-SVC-015 (Pass)
NFR-P1 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3)	qrw_core.credential; web qrw.ts	TC-CORE-070 (Pass); TC-CORE-072 (Pass); TC-WEB-013 (Pass)
NFR-P2 (M, Implemented)	SD-1 Issuance sequence (Fig. 4.2)	issuer/app.py issue()	TC-SVC-040 (Pass)
NFR-P3 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3)	web Verifier.tsx; issuer publish retry	TC-SVC-001 (Pass); TC-SVC-026 (Pass); TC-SYS-002 (Pass)
NFR-S1 (M, Implemented)	Class diagram (Fig. 3.3)	qrw_core.keys; services/secrets.py	TC-CORE-060 (Pass)
NFR-S2 (M, Implemented)	Class diagram (Fig. 3.3)	web vault.ts	TC-WEB-014 (Pass)
NFR-S3 (M, Partial)	Deployment	web vite.config.ts basicSsl	Inspection: npm run phone serves HTTPS (Pass)
NFR-S4 (M, Implemented)	Class diagram (Fig. 3.3)	qrw_core.credential.mask_nin(); issuer nin_hash()	TC-CORE-018 (Pass); TC-SVC-020 (Pass)

Requirement ID	UML / SysML artefact	Code module	Test case ID (result)
NFR-S5 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1)	verify_credential(); shared/app.py fail_safely()	TC-CORE-004 (Pass); TC-CORE-013 (Pass); TC-CORE-014 (Pass); TC-CORE-015 (Pass); TC-CORE-019 (Pass); TC-CORE-023 (Pass); TC-CORE-024 (Pass); TC-CORE-025 (Pass); TC-CORE-035 (Pass); TC-CORE-041 (Pass); TC-CORE-042 (Pass); TC-CORE-061 (Pass); TC-CORE-063 (Pass); TC-SEC-001 (Pass); TC-SEC-002 (Pass); TC-SEC-003 (Pass); TC-SEC-004 (Pass); TC-SEC-007 (Pass); TC-SEC-008 (Pass); TC-SEC-009 (Pass); TC-SEC-010 (Pass); TC-SEC-011 (Pass); TC-SEC-012 (Pass); TC-SVC-013 (Pass); TC-SVC-025 (Pass); TC-WEB-003 (Pass); TC-WEB-005 (Pass); TC-WEB-010 (Pass)
NFR-S6 (M, Implemented)	Class diagram (Fig. 3.3)	qrw_core.audit	TC-CORE-051 (Pass); TC-CORE-052 (Pass); TC-CORE-053 (Pass); TC-SEC-006 (Pass); TC-SVC-031 (Pass)
NFR-S7 (M, Implemented)	-	shared/security.py RateLimiter; auth lockout	TC-SVC-012 (Pass); TC-SVC-016 (Pass)
NFR-S8 (M, Partial)	-	Design + procedure	Inspection: data fields in credential and Issuer DB (Pass)
NFR-U1 (S, Partial)	-	web Verifier.tsx	Inspection: Verifier verdict strings (Pass)
NFR-U2 (M, Implemented)	SM-1 Verification state machine (Fig. 3.1)	web Verifier.tsx; styles.css	TC-SYS-002 (Pass); TC-WEB-011 (Pass); Inspection: TC-SYS-002 screenshots 09-10 (Pass)
NFR-U3 (M, Implemented)	SD-2 Offline verification sequence (Fig. 4.3)	web Verifier.tsx	Demo: count taps in TC-SYS-002 flow (scan, scan answer, Yes) (Pass)
NFR-U4 (M, Implemented)	-	web styles.css .btn min-height 48px	Inspection: styles.css (Pass)
NFR-C1 (M, Implemented)	Class diagram (Fig. 3.3)	qrw_core.qr	TC-CORE-001 (Pass); TC-CORE-002 (Pass); TC-CORE-003 (Pass); TC-CORE-005 (Pass); TC-CORE-022 (Pass); TC-WEB-001 (Pass)
NFR-C2 (S, Implemented)	-	web Wallet.tsx; vault.ts	TC-WEB-015 (Pass)
NFR-C3 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4)	shared/app.py GZipMiddleware; web qrw.ts packIds()	TC-CORE-071 (Pass); TC-SVC-042 (Pass); TC-WEB-016 (Pass)
NFR-C4 (M, Implemented)	SD-3 Revocation & sync sequence (Fig. 4.4)	audit/app.py MAX_BATCH	TC-SEC-007 (Pass); TC-SVC-041 (Pass)